

22. Řízení přístupu do kritické sekce a synchronizace pomocí programovacích jazyků, monitor, problém producentů a konzumentů.

Řízení přístupu do kritické sekce a synchronizace pomocí programovacích jazyků:

JAVA

- sdílená proměnná se označuje klíčovým slovem `volatile`
- synchronizace je vcelku jednoduchá
 - Syntaxe je následující: `synchronized( object ) { object.fce() // to jest kód kritické sekce }`
- dále nabízí třídu `Semaphore`, která obsahuje synchronizační nástroje
 - `acquire()` - podobné jako `sem_wait`
 - `release()` - podobné jako `sem_signal`

Monitory

Konstrukce ve vyšším programovacím jazyce poskytující stejné služby jako semafore, ale snadněji ovladatelné.

Vyskytují se v řadě jazyků pro konkurenční programování (C++, Java, C#, ...).

Mohou být implementovány pomocí semaforů.

Monitor = SW modul (podobný objektu/třídě)

- Má lokální proměnné = sdílená data
- Funkce zpřístupňující lokální data
- Inicializační část

V monitoru (v jeho funkci) smí být v daném okamžiku pouze jediný proces

- Konstrukce monitoru tak zajišťuje vzájemné vylučování
 - k vytvoření monitoru (třídy) využijeme klíčové slovo `monitor`
 - kritická sekce se definuje uvnitř monitoru do funkce `critical_section`
- Synchronizaci lze zajistit podmínkovými proměnnými uvnitř struktury
 - z venku pak voláme jen
 - `await()` - čekání (blokování procesu)
 - `finished()` - hotovo (odblokování procesu)

Příklad KS:

```
monitor CS {  
    // deklarace sdílených proměnných  
  
    critical_section() {  
        // kód kritické sekce
```

```

    }
}

// Kód vláken:
repeat
    CS.critical_section();    // kritická sekce
    // zbytková sekce
forever

```

Příklad synchronizace:

```

monitor sync {
    bool done = false;    // proměnná určující stav akce
    cond_t condition;    // deklarace podmínkové proměnné

    finished() {          // zavolá se po vykonání akce
        done = true;
        condition_signal(condition);
    }

    await() {              // zavolá se při čekání na dokončení akce
        if (!done)
            condition_wait(condition);
    }
}

// Kód vlákna provádějícího akci, zde vypočtení hodnoty sdílené
// proměnné x:
x = calculate();          // provedení akce
sync.finished();          // signalizace dokončení akce

// Kód vlákna čekajícího na dokončení akce:
sync.await();             // pokud akce nebyla ještě provedena, bude čekat
use(x);                   // použití hodnoty sdílené proměnné x vypočtené jiným
                           // vláknem

```

Problém producentů a konzumentů

- dva typy procesů:
 - producenti – produkují výrobky (data)
 - konzumenti – spotřebovávají výrobky (data)
- pro lepší efektivitu je potřeba zavést mezisklad - vyrovnávací paměť na vyprodukovaná data, nevzniknou tak zbytečné čekací doby
 - pro řízení lze využít semaforey
 - kapacita skladu – pro počet výrobků na skladě, možnost vstupu do skladu